

Quoridor – Python

Section Architecture

Table des matières

1	Architecture du projet	1
1.1	Vue d'ensemble	1
1.2	Découpage modulaire	1
1.3	Dépendances entre modules	3
1.4	Principales interfaces internes	3
1.5	Lien avec les structures de données et les algorithmes	3
1.6	Justification des choix	4
1.7	Bilan architectural	4

1 Architecture du projet

L'architecture de **Quoridor** repose sur une organisation en couches. Ce choix est adapté à la nature du logiciel : le projet ne se limite pas à une interface unique, mais combine un CLI interactif, un mode contest, une GUI GTK et un mode réseau client/serveur. Dans ce contexte, la priorité architecturale n'est pas de structurer l'application autour d'un paradigme graphique particulier, mais de centraliser la logique métier afin qu'elle soit réutilisable depuis plusieurs points d'entrée.

L'objectif principal de cette architecture est double. D'une part, il faut isoler le coeur du jeu afin que les règles, les structures de données et les algorithmes puissent être testés indépendamment des interfaces. D'autre part, il faut permettre à plusieurs modes d'exécution de partager les mêmes services applicatifs sans dupliquer la logique de validation, de persistance ou d'intelligence artificielle. Cette organisation répond donc à une exigence de cohérence fonctionnelle autant qu'à une exigence de maintenabilité.

1.1 Vue d'ensemble

Le point d'entrée du programme est le module `quoridor.__main__`, qui délègue l'exécution au CLI principal. Celui-ci interprète les options, charge la configuration, initialise l'internationalisation, puis redirige l'exécution vers le bon sous-système. L'architecture peut être résumée en six blocs principaux :

- les **interfaces**, qui collectent les actions utilisateur et affichent les résultats ;
- la **couche applicative**, qui orchestre les cas d'usage ;
- le **noyau métier**, qui contient l'état du jeu et les objets de domaine ;
- la **couche règles**, qui encode les contraintes de Quoridor ;
- les **services techniques transverses**, comme la configuration, l'internationalisation et la persistance ;
- l'**infrastructure réseau et IA**, qui étend l'application sans modifier le coeur du jeu.

La Fig. 1 présente cette organisation générale.

1.2 Découpage modulaire

Couche interfaces. Cette couche regroupe tous les points d'entrée visibles par l'utilisateur. Le module `cli.py` sert de routeur principal entre le mode interactif, le mode contest, le mode serveur et la GUI. Le module `cli_shell.py` porte la boucle interactive historique, mais il a été progressivement

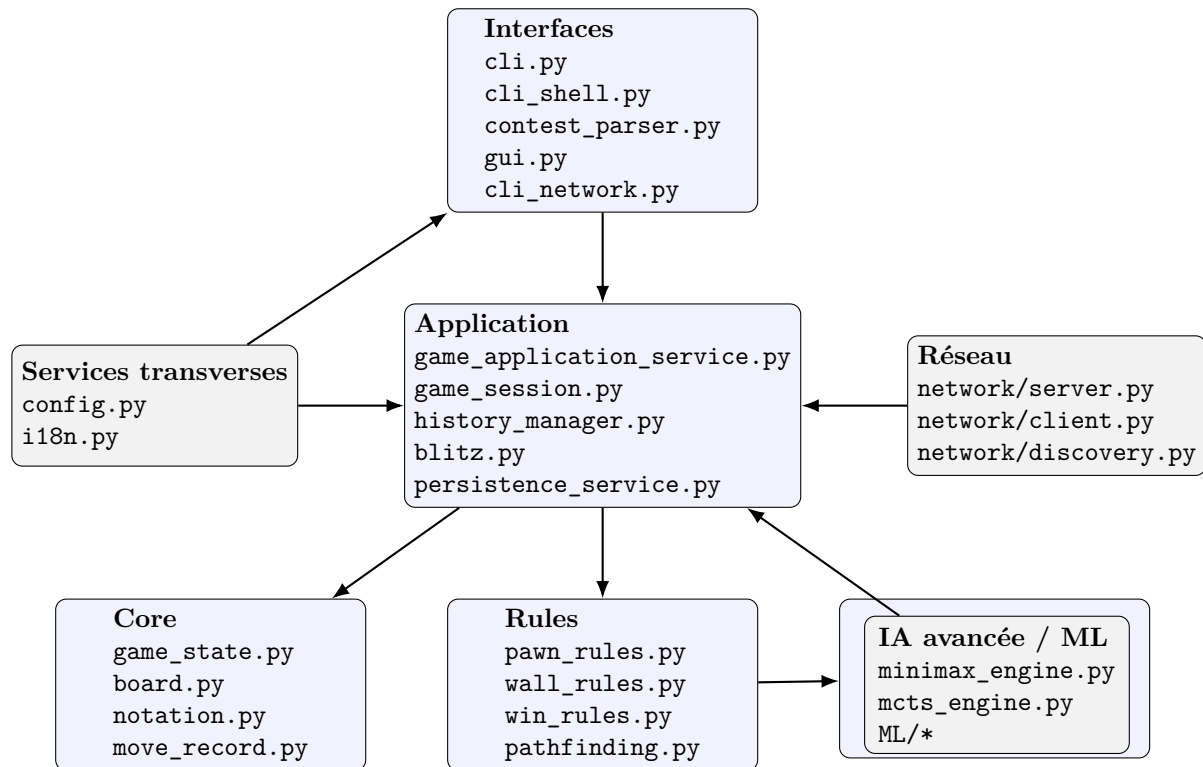


FIGURE 1 – Diagramme UML de packages simplifié de l'architecture de Quoridor

factorisé à l'aide du sous-paquet `interfaces.shell`, qui regroupe des responsabilités spécialisées : amorçage de session, état courant du shell, gestion du blitz, commandes locales, commandes réseau et événements. La GUI est concentrée dans `gui.py`, qui partage la logique métier avec le CLI via les services applicatifs plutôt que de réimplémenter les règles.

Couche application. Cette couche orchestre les cas d'usage complets. La classe `GameSession` encapsule la dynamique d'une partie : application d'un coup, alternance des joueurs, calcul de vainqueur, historique, undo/redo et intégration du blitz. La classe `GameApplicationService` joue le rôle de façade : elle expose aux interfaces des opérations de haut niveau comme charger, sauvegarder, demander un conseil, jouer un coup au format texte ou créer une nouvelle session. Cette séparation limite fortement le couplage entre l'interface et le coeur du jeu.

Couche core. Le noyau métier repose principalement sur `GameState`, qui constitue la source de vérité unique pour l'état courant d'une partie. Cette structure regroupe la taille du plateau, le joueur courant, les positions des pions, les murs restants, les murs posés, ainsi que les joueurs temporairement inactifs en mode réseau. Cette couche contient également la notation, les snapshots et les objets nécessaires à la sérialisation de l'historique.

Couche rules. Les règles du jeu sont isolées dans des modules dédiés : déplacement des pions, validation des murs, victoire et recherche de chemin. Cette couche est la plus proche du domaine pur : elle est largement stateless et déterministe. Elle est donc particulièrement testable et réutilisable. Elle manipule directement la structure de graphe sous-jacente, ce qui fait le lien naturel entre architecture et structures de données.

Couche utils. Le module `utils.graph` fournit la structure de graphe utilisée pour représenter la connectivité du plateau. Le plateau est modélisé comme une liste d'adjacence dont les arêtes sont supprimées lors de la pose des murs. Cette structure technique est volontairement maintenue en dehors des règles elles-mêmes, afin de rendre son implémentation réutilisable.

Infrastructure réseau. Le sous-paquet **network** implémente un client, un serveur multi-clients et un mécanisme de découverte UDP des serveurs locaux. D'un point de vue architectural, il ne constitue pas un second moteur de jeu : il s'appuie sur la même logique métier que le mode local. Le serveur utilise lui aussi les objets **GameSession** et les services applicatifs pour valider les coups et maintenir les parties.

Infrastructure IA et ML. Les moteurs **minimax** et **mcts** appartiennent à la couche applicative car ils exploitent le modèle métier pour produire des coups. Le sous-dossier **ML** contient les artefacts liés à l'entraînement et à l'extraction de caractéristiques pour le sélecteur ML utilisé avec MCTS. Même si cette partie reste moins mature que le reste du projet, elle est bien identifiée comme une extension algorithmique du coeur du jeu.

1.3 Dépendances entre modules

La direction générale des dépendances est descendante. Les interfaces dépendent de la couche applicative, la couche applicative dépend du noyau métier et des règles, et les règles s'appuient elles-mêmes sur les structures de données techniques. Cette orientation est saine car elle évite qu'une règle du jeu ne dépende d'un choix d'affichage, d'une entrée clavier ou d'une fenêtre graphique.

Le module **GameApplicationService** est le principal point de couture de cette architecture. Il masque la complexité interne du moteur à l'interface appelante. Ainsi, la GUI et le CLI invoquent les mêmes méthodes pour charger une partie, sauvegarder, jouer un coup ou demander un conseil. Cette façade limite la dispersion de la logique métier et facilite l'introduction de tests fonctionnels communs.

Certaines dépendances méritent cependant d'être discutées de manière critique. Tout d'abord, le shell interactif historique reste un composant volumineux, même après factorisation. Ensuite, la persistance dépend encore d'un parseur de mode contest situé dans la couche **interfaces**, ce qui n'est pas idéal d'un point de vue purement architectural. Enfin, le moteur MCTS importe directement le sélecteur ML au runtime, ce qui crée un couplage plus fort que nécessaire entre l'algorithme standard et son extension d'apprentissage.

1.4 Principales interfaces internes

Plutôt que de décrire exhaustivement toutes les fonctions, il est plus utile de mettre en avant les interfaces internes qui structurent réellement le projet.

- **GameSession** : interface centrale pour appliquer les coups, changer de tour, manipuler l'historique et calculer le gagnant. Cette classe expose notamment les opérations `play_pawn_move_from_to()`, `place_wall()`, `undo()`, `redo()` et `compute_ai_move()`.
- **GameApplicationService** : façade applicative exposée au CLI et à la GUI. Elle fournit des points d'entrée cohérents comme `load()`, `save()`, `hint()`, `play_pawn_move_token()` et `place_wall_token()`.
- **NetworkServer** / **NetworkClient** : interface du mode réseau, responsable du protocole et du transport, mais pas des règles métier elles-mêmes.
- **GameState** : représentation canonique de l'état du jeu, utilisée par toutes les couches internes.
- **ConfigManager** et **setup_i18n** : services transverses partagés par plusieurs interfaces.

Ce choix d'interfaces est important d'un point de vue architectural. Les interfaces utilisateur ne manipulent pas directement les structures profondes du domaine ; elles passent par des services ou des sessions explicitement identifiés. En pratique, cela signifie qu'une commande du shell, un clic dans la GUI ou un événement réseau peuvent déclencher des opérations différentes au niveau de l'interface, tout en convergeant vers les mêmes abstractions internes. Cette convergence améliore la cohérence fonctionnelle et réduit le risque de divergence entre les modes d'utilisation du logiciel.

La Fig. 2 illustre ces composants et leurs interactions majeures.

1.5 Flux de contrôle typique

Pour rendre l'architecture plus concrète, on peut suivre le traitement d'une commande simple du CLI, par exemple un déplacement de pion. Le shell interactif lit la commande utilisateur, la normalise,

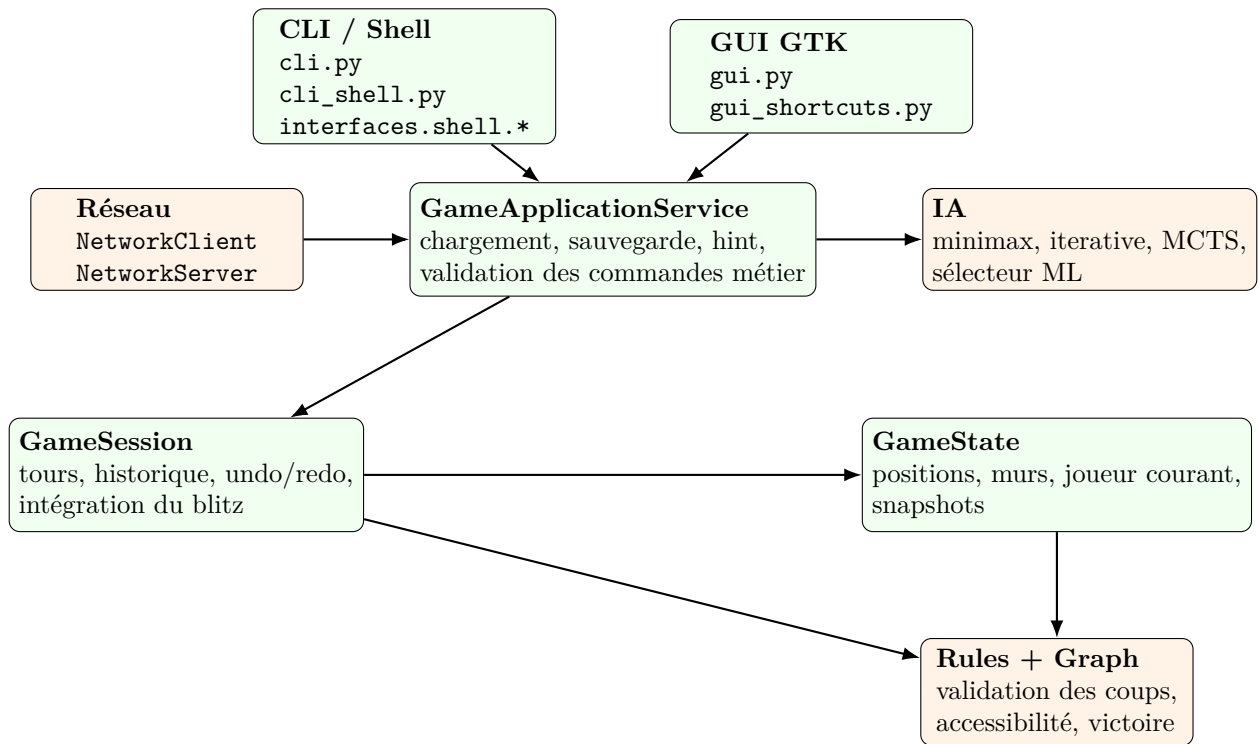


FIGURE 2 – Diagramme UML de composants simplifié et dépendances principales

puis délègue son interprétation à ses handlers spécialisés. Ceux-ci convertissent la saisie en appel vers la couche applicative ou directement vers **GameSession** selon le cas. La session consulte alors **GameState**, sollicite les modules de règles pour valider le coup, met à jour l'état, enregistre l'action dans l'historique, puis rend le contrôle à l'interface qui réaffiche l'état du jeu.

Le même principe vaut pour la GUI et pour le réseau. Dans la GUI, l'action provient d'un menu ou d'un glisser-déposer ; côté réseau, elle provient d'une commande reçue par le serveur. Dans les deux cas, la validation du coup repose sur les mêmes modules métier. Ce point est essentiel : l'architecture ne se contente pas de juxtaposer plusieurs interfaces, elle leur impose un noyau commun qui garantit un comportement homogène.

1.6 Lien avec les structures de données et les algorithmes

L'architecture choisie n'est pas indépendante des structures de données. Au contraire, elle a été construite pour leur laisser une place centrale sans les exposer directement aux interfaces. La structure de graphe utilisée pour représenter le plateau est manipulée par **GameState** et exploitée par les modules de règles. Les algorithmes de recherche de chemin, de validation des murs et de calcul des coups légaux sont donc situés là où leur utilité est la plus naturelle : au voisinage immédiat du domaine.

Les algorithmes d'intelligence artificielle utilisent ensuite ces mêmes structures de données via des services applicatifs. Minimax, iterative deepening et MCTS ne sont pas branchés directement sur une interface, mais sur l'état du jeu et les règles. Ce positionnement garantit qu'un conseil affiché en CLI, une suggestion de coup en GUI ou une décision prise en mode réseau s'appuient sur la même base algorithmique.

Ce lien explicite entre architecture et structures de données justifie le découpage retenu. Une architecture uniquement pensée autour des interfaces aurait eu tendance à disperser la logique de validation dans plusieurs composants, alors que le choix effectué place la représentation du plateau et les algorithmes de graphe au centre du système. C'est précisément ce point qui rend possible la réutilisation des mêmes règles pour le jeu local, l'undo/redo, la sauvegarde, l'IA et le mode réseau.

1.7 Justification des choix

Les choix architecturaux retenus répondent principalement à trois objectifs. Le premier est la **réutilisation** : une même logique métier doit alimenter plusieurs interfaces sans divergence de comportement. Le second est la **testabilité** : il doit être possible de tester le coeur du projet sans démarrer l'interface graphique ni le réseau. Le troisième est l'**extensibilité** : l'ajout du blitz, du réseau ou de nouvelles heuristiques IA doit rester possible sans refondre complètement le coeur du jeu.

Une autre justification importante tient à la granularité choisie. Le projet aurait pu rester organisé autour de quelques fichiers très larges, notamment pour le shell ou les services applicatifs. Au contraire, la factorisation en sous-modules spécialisés permet de limiter la taille des composants, de clarifier les responsabilités et de rendre les tests plus ciblés. Cette approche n'élimine pas complètement la complexité, mais elle la rend plus maîtrisable.

Cette architecture présente toutefois certaines limites. Le shell interactif reste historiquement central et a nécessité une factorisation progressive. L'intégration GUI n'offre pas encore la même surface fonctionnelle que le CLI, notamment pour le réseau. Enfin, la partie ML n'est pas encore aussi bien isolée que les autres briques. Ces points n'annulent pas la cohérence d'ensemble, mais ils constituent des axes d'amélioration réalistes.

1.8 Bilan architectural

Au final, l'architecture de **Quoridor** est lisible, implémentable et cohérente avec la complexité réelle du projet. Elle évite de placer les règles du jeu dans les interfaces, centralise l'état dans une structure métier identifiée, et s'appuie sur des services applicatifs communs pour partager les cas d'usage entre le CLI, la GUI et le mode réseau. C'est cette combinaison entre séparation des responsabilités, réutilisation et lien explicite avec les algorithmes du jeu qui en fait une base solide pour le projet final.